LAPORAN PRAKTIKUM

POSTTEST 7

ALGORITMA PEMROGRAMAN LANJUT

Disusun oleh:

Erfandi Marselianus(2509106098)

Kelas (C'25)

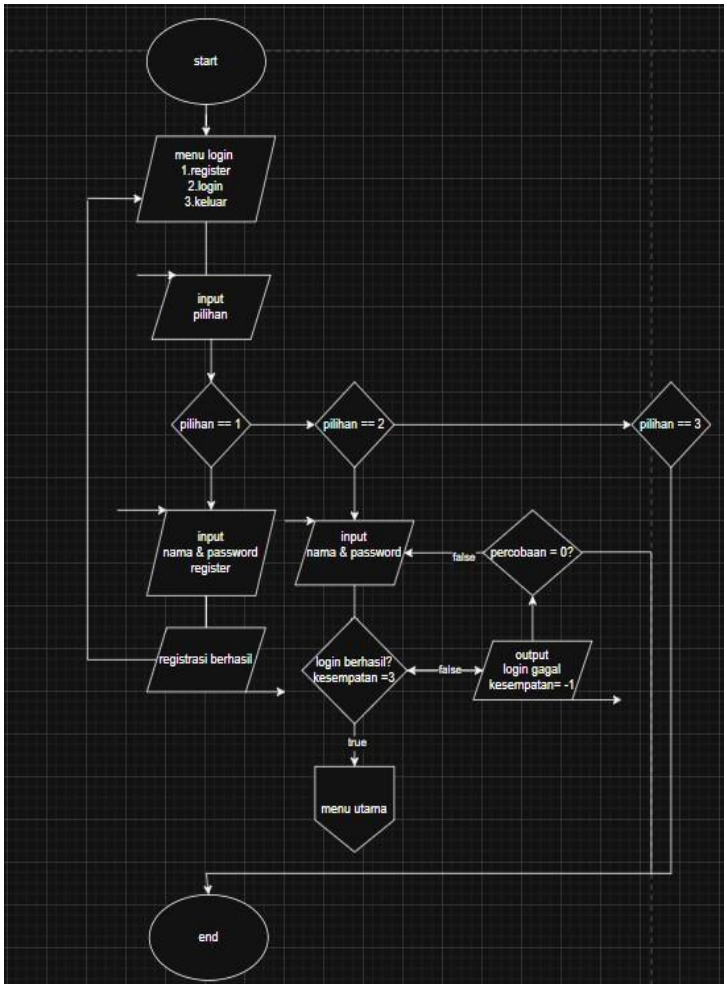
PROGRAM STUDI INFORMATIKA

UNIVERSITAS MULAWARMAN

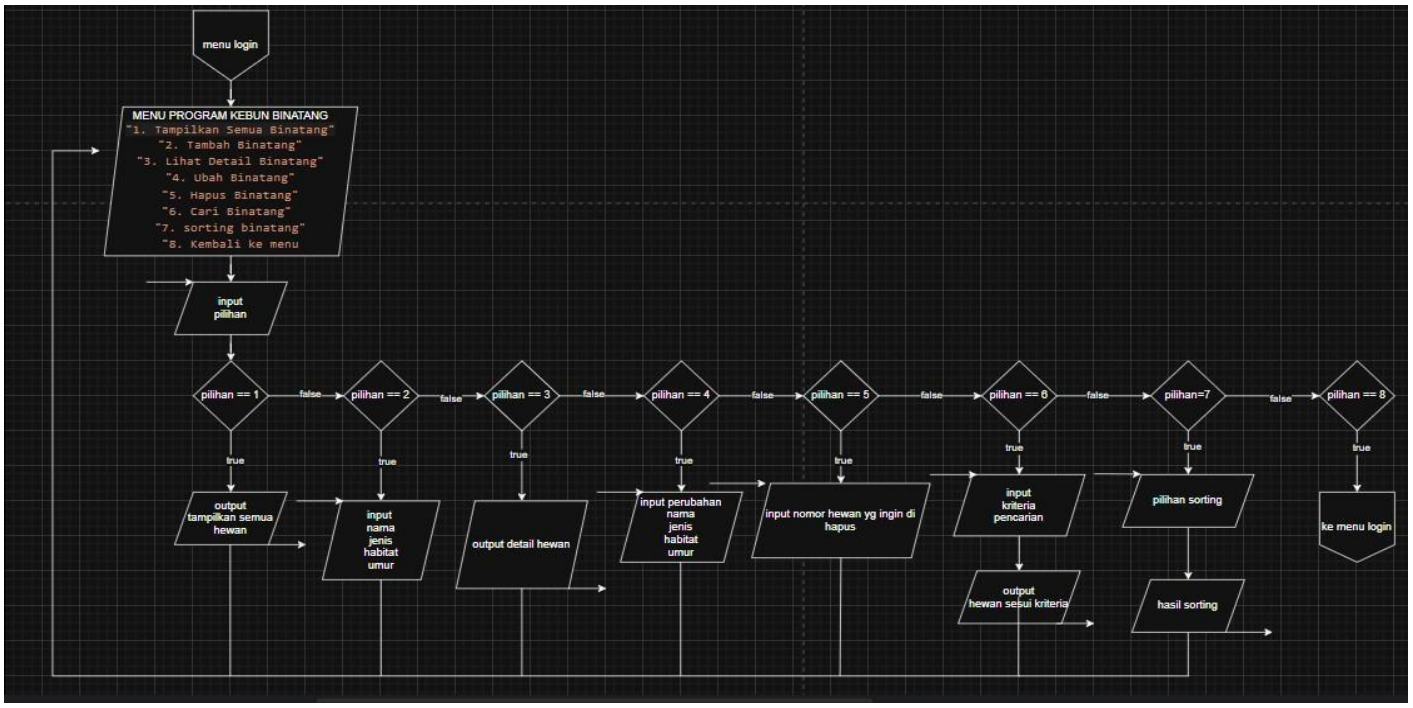
SAMARINDA

2025

1. Flowchart


start
menu login
1.register
2.login
3.keluar
input
pilihan
pilihan == 1
pilihan == 2
pilihan == 3
input
nama & password
register
input
nama & password
false
percobaan = 0?
registrasi berhasil
login berhasil?
kesempatan =3
false
output
login gagal
kesempatan= -1
true
menu utama
end

Gambar flowchart 1.1



Gambar flowchart 1.2

Penjelasan singkat flowchart

Program dimulai

Masuk menu login pilihan 1-3

1).register

2).login(kesempatan input 3 kali jika gagal/salah input)

3).keluar program

Masuk menu PROGRAM KEBUN BINATANG

Pilihan menu dari 1-7

1).Tampilkan hewan

2).Tambah Hewan

3).Lihat detail hewan

4).ubah hewan

5).Hapus hewan

6).Cari hewan berdasarkan kriteria{searching}

7).sorting Binatang

8).Ke menu utama

2. Deskripsi Singkat Program

Program ini bertujuan untuk Menampilkan,Menambah,Melihat detail,Mengubah dan Hapus Hewan kebun Binatang Dimana hewan yang di tampilkan akan menunjukan detail mulai dari nama,jenis,habitat dan umur

3. Source Code

```
void tampilkanHeader(char judul[]) {
    cout << "\n=== " << judul << " ===\n";
}

void tampilkanMenuUtama() {
    printf("\n=== MENU UTAMA ===\n");
    printf("1. Register\n");
    printf("2. Login\n");
    printf("3. Keluar\n");
    printf("Pilihan: ");
}

void tampilkanMenuKebunBinatang() {
    cout << "\n=== MENU PROGRAM KEBUN BINATANG ===" << endl;
    cout << "1. Tampilkan Semua Binatang" << endl;
    cout << "2. Tambah Binatang" << endl;
    cout << "3. Lihat Detail Binatang" << endl;
    cout << "4. Ubah Binatang" << endl;
    cout << "5. Hapus Binatang" << endl;
    cout << "6. Cari Binatang" << endl;
    cout << "7. Sorting Binatang" << endl;
    cout << "8. KEMBALI KE MENU UTAMA" << endl;
    cout << "Pilihan: ";
}
```

```
void tampilkanBinatang(Binatang b, int nomor) {
    cout << "\n--- Data Binatang ---" << endl;
    cout << "Kode    : " << b.kode << endl;
    cout << "Nama    : " << b.nama << endl;
    cout << "Jenis   : " << b.jenis << endl;
    cout << "Habitat : " << b.habitat << endl;
    cout << "Umur    : " << b.umur << " tahun" << endl;
}

void tampilkanBinatangRingkas(Binatang b, int nomorUrut) {
    cout << nomorUrut << ". [Kode: " << b.kode << "] "
         << b.nama << " (" << b.jenis << ")" << endl;
}

void validasiJenis(const char jenis[]) {
    if (strcmp(jenis, "Karnivora") != 0 &&
        strcmp(jenis, "Herbivora") != 0 &&
        strcmp(jenis, "Omnivora") != 0) {
        throw invalid_argument("Error: Jenis binatang harus Karnivora, Herbivora, atau
Omnivora!");
    }
}

void validasiHabitat(const char habitat[]) {
    if (strcmp(habitat, "Darat") != 0 &&
        strcmp(habitat, "Air") != 0 &&
        strcmp(habitat, "Udara") != 0) {
        throw invalid_argument("Error: Habitat harus Darat, Air, atau Udara!");
    }
}

void validasiUmur(int umur) {
    if (umur < 0) {
        throw runtime_error("Error: Umur tidak boleh negatif!");
    }
    if (umur > 200) {
        throw runtime_error("Error: Umur terlalu besar! Maksimal 200 tahun.");
    }
}

void validasiNama(const char nama[]) {
    if (strlen(nama) == 0) {
        throw length_error("Error: Nama binatang tidak boleh kosong!");
    }
}

int CariKode(Binatang *arr, int kiri, int kanan, int targetKode) {
    if (kanan >= kiri) {
        int tengah = kiri + (kanan - kiri) / 2;

        if ((arr + tengah)->kode == targetKode) {
            return tengah;
        }

        if ((arr + tengah)->kode > targetKode) {
```

```
            return CariKode(arr, kiri, tengah - 1, targetKode);
        }

        return CariKode(arr, tengah + 1, kanan, targetKode);
    }
    return -1;
}

void cariBerdasarkanKode(Binatang *arr, int *n, int targetKode) {
    if (targetKode <= 0) {
        throw invalid_argument("Error: Kode binatang harus bernilai positif!");
    }

    if (*n == 0) {
        throw runtime_error("Error: Belum ada data binatang untuk dicari!");
    }

    Binatang tempArray[MAX_binatang];
    for(int i = 0; i < *n; i++) {
        tempArray[i] = *(arr + i);
    }

    for(int i = 0; i < *n - 1; i++) {
        for(int j = 0; j < *n - i - 1; j++) {
            if(tempArray[j].kode > tempArray[j+1].kode) {
                Binatang temp = tempArray[j];
                tempArray[j] = tempArray[j+1];
                tempArray[j+1] = temp;
            }
        }
    }

    int hasil = CariKode(tempArray, 0, *n - 1, targetKode);

    if (hasil != -1) {
        cout << "\n>>> HASIL: Kode " << targetKode << " ditemukan! <<<" << endl;
        cout << "(Menggunakan metode Binary Search)" << endl;
        tampilkanBinatang(tempArray[hasil], hasil+1);
    } else {
        cout << "\n>>> HASIL: Kode " << targetKode << " TIDAK ditemukan! <<<" << endl;
        cout << "(Menggunakan metode Binary Search)" << endl;
    }
}

int CariJenis(Binatang *arr, int *n, char targetJenis[]) {
    if (*n == 0) return -1;

    Binatang tempArray[MAX_binatang];
    for(int i = 0; i < *n; i++) {
        tempArray[i] = *(arr + i);
    }

    cout << "\n[Mengurutkan data berdasarkan JENIS]" << endl;
    for(int i = 0; i < *n - 1; i++) {
        for(int j = 0; j < *n - i - 1; j++) {
            if(strcmp(tempArray[j].jenis, tempArray[j+1].jenis) > 0) {
```

```
                Binatang temp = tempArray[j];
                tempArray[j] = tempArray[j+1];
                tempArray[j+1] = temp;
            }
        }
    }

    cout << "Data setelah diurutkan berdasarkan jenis:" << endl;
    for(int i = 0; i < *n; i++) {
        cout << "Indeks " << i << ": Jenis = " << tempArray[i].jenis
             << ", Nama = " << tempArray[i].nama << endl;
    }

    int step = sqrt(*n);
    int prev = 0;

    cout << "\n[Proses Mencari]" << endl;
    cout << "Ukuran lompatan = " << *n << " = " << step << endl;
    cout << "Mencari jenis: '" << targetJenis << "'" << endl;

    int hitungLompatan = 0;
    while (prev < *n && strcmp(tempArray[min(step, *n) - 1].jenis, targetJenis) < 0) {
        prev = step;
        step += sqrt(*n);
        hitungLompatan++;
        cout << "Lompatan ke-" << hitungLompatan << ": prev = " << prev << ", step = "
<< step << endl;
        if (prev >= *n) {
            cout << "Melewati batas array, tidak ditemukan dalam blok" << endl;
            return -1;
        }
    }

    cout << "\nMelakukan Linear Search dalam blok [" << prev << " ... " << min(step, *n)
<< "]" << endl;
    int hitungLinear = 0;
    while (prev < min(step, *n)) {
        hitungLinear++;
        cout << "  Memeriksa indeks " << prev << ": " << tempArray[prev].jenis;
        if (strcmp(tempArray[prev].jenis, targetJenis) == 0) {
            cout << " -> COCOK" << endl;
            cout << "\n>>> Ditemukan pada indeks ke-" << prev << "! <<<" << endl;
            cout << "(Menggunakan metode Jump Search)" << endl;
            cout << "Jumlah lompatan: " << hitungLompatan << ", Linear check: " <<
hitungLinear << endl;
            return prev;
        }
        cout << " -> TIDAK COCOK" << endl;
        prev++;
    }

    return -1;
}

void cariBerdasarkanJenis(Binatang *arr, int *n, char targetJenis[]) {
    if (*n == 0) {
```

```
        throw runtime_error("Error: Belum ada data binatang untuk dicari!");
    }

    int hasil = CariJenis(arr, n, targetJenis);

    if (hasil != -1) {
        cout << "\n=== SEMUA BINATANG DENGAN JENIS '" << targetJenis << "' ===" << endl;

        Binatang tempArray[MAX_binatang];
        for(int i = 0; i < *n; i++) {
            tempArray[i] = *(arr + i);
        }

        for(int i = 0; i < *n - 1; i++) {
            for(int j = 0; j < *n - i - 1; j++) {
                if(strcmp(tempArray[j].jenis, tempArray[j+1].jenis) > 0) {
                    Binatang temp = tempArray[j];
                    tempArray[j] = tempArray[j+1];
                    tempArray[j+1] = temp;
                }
            }
        }

        int ditemukan = 0;
        for(int i = 0; i < *n; i++) {
            if(strcmp(tempArray[i].jenis, targetJenis) == 0) {
                tampilkanBinatang(tempArray[i], i+1);
                ditemukan++;
            }
        }
        cout << "\nTotal ditemukan: " << ditemukan << " binatang" << endl;
    } else {
        cout << "\n>>> HASIL: Jenis '" << targetJenis << "' TIDAK ditemukan! <<<" <<
endl;
        cout << "(Menggunakan metode Jump Search - O(√n))" << endl;
    }
}

int CariHabitat(Binatang *arr, int *n, char targetHabitat[]) {
    if (*n == 0) return -1;

    Binatang tempArray[MAX_binatang];
    for(int i = 0; i < *n; i++) {
        tempArray[i] = *(arr + i);
    }

    cout << "\n[Mengurutkan hewan berdasarkan HABITAT]" << endl;
    for(int i = 0; i < *n - 1; i++) {
        for(int j = 0; j < *n - i - 1; j++) {
            if(strcmp(tempArray[j].habitat, tempArray[j+1].habitat) > 0) {
                Binatang temp = tempArray[j];
                tempArray[j] = tempArray[j+1];
                tempArray[j+1] = temp;
            }
        }
    }
```

```
    cout << "Data setelah diurutkan berdasarkan habitat:" << endl;
    for(int i = 0; i < *n; i++) {
        cout << "Indeks " << i << ": Habitat = " << tempArray[i].habitat
             << ", Nama = " << tempArray[i].nama << endl;
    }

    int low = 0;
    int high = *n - 1;
    int pos;
    int hitungTebakan = 0;

    cout << "\n[Proses]" << endl;
    cout << "Mencari habitat: '" << targetHabitat << "'" << endl;

    int targetValue = 0;
    for(int i = 0; targetHabitat[i] != '\0'; i++) {
        targetValue += targetHabitat[i];
    }

    while (low <= high) {
        int lowValue = 0, highValue = 0;
        for(int i = 0; tempArray[low].habitat[i] != '\0'; i++) lowValue +=
tempArray[low].habitat[i];
        for(int i = 0; tempArray[high].habitat[i] != '\0'; i++) highValue +=
tempArray[high].habitat[i];

        if (lowValue == highValue) {
            if (targetValue == lowValue) pos = low;
            else break;
        } else {
            pos = low + ((targetValue - lowValue) * (high - low) / (highValue -
lowValue));
        }

        hitungTebakan++;
        cout << "Tebakan ke-" << hitungTebakan << ": pos = " << pos;

        if (pos < 0 || pos > high) {
            cout << " (di luar batas)" << endl;
            break;
        }

        cout << ", Habitat = " << tempArray[pos].habitat << endl;

        if (strcmp(tempArray[pos].habitat, targetHabitat) == 0) {
            cout << "\n>>> Ditemukan pada indeks ke-" << pos << "! <<<" << endl;
            cout << "(Menggunakan metode Interpolation Search)" << endl;
            cout << "Jumlah tebakan: " << hitungTebakan << endl;
            return pos;
        }

        if (strcmp(tempArray[pos].habitat, targetHabitat) < 0) {
            cout << "  -> Terlalu kecil, cari di kanan" << endl;
            low = pos + 1;
        } else {
```

```
            cout << "  -> Terlalu besar, cari di kiri" << endl;
            high = pos - 1;
        }
    }

    return -1;
}

void cariBerdasarkanHabitat(Binatang *arr, int *n, char targetHabitat[]) {
    if (*n == 0) {
        throw runtime_error("Error: Belum ada data binatang untuk dicari!");
    }

    int hasil = CariHabitat(arr, n, targetHabitat);

    if (hasil != -1) {
        cout << "\n=== SEMUA BINATANG DENGAN HABITAT '" << targetHabitat << "' ===" <<
endl;

        Binatang tempArray[MAX_binatang];
        for(int i = 0; i < *n; i++) {
            tempArray[i] = *(arr + i);
        }

        for(int i = 0; i < *n - 1; i++) {
            for(int j = 0; j < *n - i - 1; j++) {
                if(strcmp(tempArray[j].habitat, tempArray[j+1].habitat) > 0) {
                    Binatang temp = tempArray[j];
                    tempArray[j] = tempArray[j+1];
                    tempArray[j+1] = temp;
                }
            }
        }

        int ditemukan = 0;
        for(int i = 0; i < *n; i++) {
            if(strcmp(tempArray[i].habitat, targetHabitat) == 0) {
                tampilkanBinatang(tempArray[i], i+1);
                ditemukan++;
            }
        }
        cout << "\nTotal ditemukan: " << ditemukan << " binatang" << endl;
    } else {
        cout << "\n>>> HASIL: Habitat '" << targetHabitat << "' TIDAK ditemukan! <<<" <<
endl;
        cout << "(Menggunakan metode Interpolation Search )" << endl;
    }
}

void cariBerdasarkanNama(Binatang *arr, int *n, char keyword[]) {
    if (*n == 0) {
        throw runtime_error("Error: Belum ada data binatang untuk dicari!");
    }

    if (strlen(keyword) == 0) {
        throw invalid_argument("Error: Kata kunci pencarian tidak boleh kosong!");
```

```
    }

    cout << "\n[Proses mencari]" << endl;
    int ditemukan = 0;

    for(int i = 0; i < *n; i++) {
        if(strstr((arr + i)->nama, keyword) != NULL) {
            if(ditemukan == 0) {
                cout << "\n>>> HASIL PENCARIAN <<<" << endl;
            }
            tampilkanBinatang(*(arr + i), i+1);
            ditemukan++;
        }
    }

    if(ditemukan == 0) {
        cout << "\nTidak ada binatang dengan nama yang mengandung '" << keyword << "'"
<< endl;
    } else {
        cout << "\nDitemukan " << ditemukan << " binatang." << endl;
    }
}

void cariBinatang(Binatang kebunBinatang[], int banyak) {
    if(banyak > 0) {
        tampilkanHeader("CARI BINATANG");
        cout << "Pilih kriteria pencarian:" << endl;
        cout << "1. Cari berdasarkan kode" << endl;
        cout << "2. Cari berdasarkan nama" << endl;
        cout << "3. Cari berdasarkan jenis" << endl;
        cout << "4. Cari berdasarkan habitat" << endl;
        cout << "Pilihan: ";

        int kriteria;
        cin >> kriteria;

        try {
            switch(kriteria) {
                case 1: {
                    int kodeCari;
                    cout << "Masukkan kode hewan: ";
                    cin >> kodeCari;
                    cariBerdasarkanKode(kebunBinatang, &banyak, kodeCari);
                    break;
                }
                case 2: {
                    char keyword[50];
                    cout << "Masukkan nama binatang: ";
                    cin >> keyword;
                    cariBerdasarkanNama(kebunBinatang, &banyak, keyword);
                    break;
                }
                case 3: {
                    char keyword[50];
                    cout << "Masukkan jenis binatang (Karnivora/Herbivora/Omnivora): ";
                    cin >> keyword;
```

```
                    cariBerdasarkanJenis(kebunBinatang, &banyak, keyword);
                    break;
                }
                case 4: {
                    char keyword[50];
                    cout << "Masukkan habitat (Darat/Air/Udara): ";
                    cin >> keyword;
                    cariBerdasarkanHabitat(kebunBinatang, &banyak, keyword);
                    break;
                }
                default:
                    throw invalid_argument("Error: Pilihan tidak valid! Silakan pilih 1-
4.");
            }
        }
        catch (const invalid_argument& e) {
            cout << "\nKESALAHAN DITEMUKAN " << e.what() << endl;
        }
        catch (const runtime_error& e) {
            cout << "\nKESALAHAN DITEMUKAN " << e.what() << endl;
        }
        catch (const length_error& e) {
            cout << "\nKESALAHAN DITEMUKAN! " << e.what() << endl;
        }
        catch (...) {
            cout << "\nTerjadi kesalahan yang tidak diketahui!" << endl;
        }
    } else {
        cout << "\nTidak ada data binatang." << endl;
    }
}

void SortNama(Binatang arr[], int n) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(strcmp(arr[j].nama, arr[j+1].nama) < 0) {
                Binatang temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
    cout << "\n=== SORTING NAMA SELESAI ===" << endl;
}

void SortUmur(Binatang arr[], int n) {
    for(int i = 1; i < n; i++) {
        Binatang key = arr[i];
        int j = i - 1;

        while(j >= 0 && arr[j].umur > key.umur) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
```

```
    cout << "\n=== SORTING UMUR SELESAI ===" << endl;
}

void SortHabitat(Binatang arr[], int n) {
    for(int i = 0; i < n - 1; i++) {
        int indeksMin = i;

        for(int j = i + 1; j < n; j++) {
            if(strcmp(arr[j].habitat, arr[indeksMin].habitat) < 0) {
                indeksMin = j;
            }
        }

        if(indeksMin != i) {
            Binatang temp = arr[i];
            arr[i] = arr[indeksMin];
            arr[indeksMin] = temp;
        }
    }
    cout << "\n=== SORTING HABITAT SELESAI ===" << endl;
}

void SortKode(Binatang arr[], int n) {
    for(int i = 0; i < n - 1; i++) {
        for(int j = 0; j < n - i - 1; j++) {
            if(arr[j].kode > arr[j+1].kode) {
                Binatang temp = arr[j];
                arr[j] = arr[j+1];
                arr[j+1] = temp;
            }
        }
    }
    cout << "\n=== SORTING KODE SELESAI ===" << endl;
}

void menuSorting() {
    if(banyak == 0) {
        cout << "\nBelum ada data binatang untuk di-sort!" << endl;
        return;
    }

    int pilihanSort;
    cout << "\n=== MENU SORTING BINATANG ===" << endl;
    cout << "1. Kode (Ascending)" << endl;
    cout << "2. Nama (Z-A)" << endl;
    cout << "3. Umur (Termuda ke Tertua)" << endl;
    cout << "4. Habitat (A-Z)" << endl;
    cout << "5. Kembali" << endl;
    cout << "Pilihan: ";
    cin >> pilihanSort;

    Binatang tempArray[MAX_binatang];
    for(int i = 0; i < banyak; i++) {
        tempArray[i] = kebunBinatang[i];
    }
```

```
    switch(pilihanSort) {
        case 1:
            SortKode(tempArray, banyak);
            cout << "\n=== HASIL SORTING/PENGURUTAN ===" << endl;
            for(int i = 0; i < banyak; i++) {
                tampilkanBinatang(tempArray[i], i+1);
            }

            char simpan;
            cout << "\nSimpan hasil sorting ke data utama? (y/n): ";
            cin >> simpan;
            if(simpan == 'y' || simpan == 'Y') {
                for(int i = 0; i < banyak; i++) {
                    kebunBinatang[i] = tempArray[i];
                }
                cout << "Data berhasil disimpan!" << endl;
            }
            break;

        case 2:
            SortNama(tempArray, banyak);
            cout << "\n=== HASIL SORTING/PENGURUTAN ===" << endl;
            for(int i = 0; i < banyak; i++) {
                tampilkanBinatang(tempArray[i], i+1);
            }

            cout << "\nSimpan hasil sorting ke data utama? (y/n): ";
            cin >> simpan;
            if(simpan == 'y' || simpan == 'Y') {
                for(int i = 0; i < banyak; i++) {
                    kebunBinatang[i] = tempArray[i];
                }
                cout << "Data berhasil disimpan!" << endl;
            }
            break;

        case 3:
            SortUmur(tempArray, banyak);
            cout << "\n=== HASIL SORTING/PENGURUTAN ===" << endl;
            for(int i = 0; i < banyak; i++) {
                tampilkanBinatang(tempArray[i], i+1);
            }

            cout << "\nSimpan hasil sorting ke data utama? (y/n): ";
            cin >> simpan;
            if(simpan == 'y' || simpan == 'Y') {
                for(int i = 0; i < banyak; i++) {
                    kebunBinatang[i] = tempArray[i];
                }
                cout << "Data berhasil disimpan!" << endl;
            }
            break;

        case 4:
            SortHabitat(tempArray, banyak);
            cout << "\n=== HASIL SORTING/PENGURUTAN ===" << endl;
```

```
            for(int i = 0; i < banyak; i++) {
                tampilkanBinatang(tempArray[i], i+1);
            }

            cout << "\nSimpan hasil sorting ke data utama? (y/n): ";
            cin >> simpan;
            if(simpan == 'y' || simpan == 'Y') {
                for(int i = 0; i < banyak; i++) {
                    kebunBinatang[i] = tempArray[i];
                }
                cout << "Data berhasil disimpan!" << endl;
            }
            break;

        case 5:
            cout << "Kembali ke menu utama..." << endl;
            break;

        default:
            cout << "Pilihan tidak valid!" << endl;
    }
}

void registrasi(Akun *user) {
    tampilkanHeader("REGISTRASI");
    printf("Masukkan Nama: ");
    scanf("%s", (*user).nama);
    printf("Masukkan Password: ");
    scanf("%s", (*user).passnim);
    printf("\nRegistrasi Berhasil!\n");
}

void login(Akun user, char loginNama[], char loginPassnim[], int &kesempatan, bool
&berhasil) {
    if(strcmp(loginNama, user.nama) == 0 && strcmp(loginPassnim, user.passnim) == 0) {
        printf("\nLogin Berhasil!\n");
        berhasil = true;
    } else {
        kesempatan--;
        printf("Login Gagal! Sisa kesempatan: %d\n", kesempatan);
        if(kesempatan > 0) {
            printf("Silakan coba lagi.\n");
        }
        berhasil = false;
    }
}

void tampilkanSemuaBinatang(Binatang kebunBinatang[], int banyak) {
    if(banyak == 0) {
        cout << "\nBelum ada data binatang." << endl;
        return;
    }

    tampilkanHeader("DAFTAR SEMUA BINATANG");
    cout << "Total: " << banyak << " binatang" << endl;
    for(int i = 0; i < banyak; i++) {
```

```
        tampilkanBinatangRingkas(kebunBinatang[i], i+1);
    }
}

void tambahBinatang(Binatang kebunBinatang[], int *banyak) {
    if(*banyak < MAX_binatang) {
        Binatang b;

        tampilkanHeader("TAMBAH BINATANG BARU");

        kodeTerakhir++;
        b.kode = kodeTerakhir;
        cout << "Kode hewan : " << b.kode << endl;

        try {
            cout << "Masukkan nama binatang: ";
            cin >> b.nama;
            validasiNama(b.nama);

            cout << "Masukkan jenis (Karnivora/Herbivora/Omnivora): ";
            cin >> b.jenis;
            validasiJenis(b.jenis);

            cout << "Masukkan habitat (Darat/Air/Udara): ";
            cin >> b.habitat;
            validasiHabitat(b.habitat);

            cout << "Masukkan umur (tahun): ";
            cin >> b.umur;
            validasiUmur(b.umur);

            kebunBinatang[*banyak] = b;
            (*banyak)++;

            cout << "\nBinatang berhasil ditambahkan!" << endl;
            cout << "Kode hewan: " << b.kode << endl;
            tampilkanBinatang(b, *banyak);
        }
        catch (const invalid_argument& e) {
            cout << "\nTERJADI KESALAHAN " << e.what() << endl;
            cout << "Data binatang GAGAL ditambahkan!" << endl;
        }
        catch (const runtime_error& e) {
            cout << "\nTERJADI KESALAHAN! " << e.what() << endl;
            cout << "Data binatang GAGAL ditambahkan!" << endl;
        }
        catch (const length_error& e) {
            cout << "\nTERJADI KESALAHAN! " << e.what() << endl;
            cout << "Data binatang GAGAL ditambahkan!" << endl;
        }
        catch (...) {
            cout << "\nTerjadi kesalahan yang tidak diketahui saat menambah data!" <<
endl;
        }
    } else {
        cout << "\nData binatang sudah penuh!" << endl;
```

```
    }
}

void lihatDetailBinatang(Binatang kebunBinatang[], int banyak) {
    if(banyak > 0) {
        int pilihan;
        cout << "\n=== LIHAT DETAIL BINATANG ===" << endl;
        cout << "1. Cari berdasarkan nomor urut" << endl;
        cout << "2. Cari berdasarkan kode hewan" << endl;
        cout << "Pilihan: ";
        cin >> pilihan;

        try {
            if(pilihan == 1) {
                int index;
                cout << "Masukkan nomor urut (1-" << banyak << "): ";
                cin >> index;

                if(index < 1 || index > banyak) {
                    throw out_of_range("Error: Nomor urut di luar batas array!");
                }

                tampilkanBinatang(kebunBinatang[index-1], index);
            }
            else if(pilihan == 2) {
                int kodeCari;
                cout << "Masukkan kode hewan: ";
                cin >> kodeCari;

                if(kodeCari <= 0) {
                    throw invalid_argument("Error: Kode hewan harus positif!");
                }

                bool ditemukan = false;
                for(int i = 0; i < banyak; i++) {
                    if(kebunBinatang[i].kode == kodeCari) {
                        tampilkanBinatang(kebunBinatang[i], i+1);
                        ditemukan = true;
                        break;
                    }
                }

                if(!ditemukan) {
                    cout << "Hewan dengan kode " << kodeCari << " tidak ditemukan!" <<
endl;
                }
            }
            else {
                throw invalid_argument("Error: Pilihan tidak valid!");
            }
        }
        catch (const invalid_argument& e) {
            cout << "\nTERJADI KESALAHAN " << e.what() << endl;
        }
        catch (const out_of_range& e) {
            cout << "\nTERJADI KESALAHAN " << e.what() << endl;
```

```
        }
        catch (...) {
            cout << "\nTerjadi kesalahan yang tidak diketahui!" << endl;
        }
    } else {
        cout << "\nTidak ada data binatang." << endl;
    }
}

void ubahBinatang(Binatang kebunBinatang[], int banyak) {
    if(banyak > 0) {
        int pilihan;
        cout << "\n=== UBAH DATA BINATANG ===" << endl;
        cout << "1. Ubah berdasarkan nomor urut" << endl;
        cout << "2. Ubah berdasarkan kode hewan" << endl;
        cout << "Pilihan: ";
        cin >> pilihan;

        int index = -1;

        try {
            if(pilihan == 1) {
                int nomor;
                cout << "Masukkan nomor urut (1-" << banyak << "): ";
                cin >> nomor;

                if(nomor < 1 || nomor > banyak) {
                    throw out_of_range("Error: Nomor urut di luar batas!");
                }
                index = nomor - 1;
            }
            else if(pilihan == 2) {
                int kodeCari;
                cout << "Masukkan kode hewan: ";
                cin >> kodeCari;

                if(kodeCari <= 0) {
                    throw invalid_argument("Error: Kode hewan harus positif!");
                }

                for(int i = 0; i < banyak; i++) {
                    if(kebunBinatang[i].kode == kodeCari) {
                        index = i;
                        break;
                    }
                }

                if(index == -1) {
                    cout << "Hewan dengan kode " << kodeCari << " tidak ditemukan!" <<
endl;
                    return;
                }
            }
            else {
                throw invalid_argument("Error: Pilihan tidak valid!");
            }
```

```
            if(index != -1) {
                Binatang &b = kebunBinatang[index];

                cout << "\nData lama:" << endl;
                tampilkanBinatang(b, index+1);

                cout << "\nMasukkan data baru:" << endl;
                cout << "Nama binatang (" << b.nama << "): ";
                cin >> b.nama;
                validasiNama(b.nama);

                cout << "Jenis (" << b.jenis << "): ";
                cin >> b.jenis;
                validasiJenis(b.jenis);

                cout << "Habitat (" << b.habitat << "): ";
                cin >> b.habitat;
                validasiHabitat(b.habitat);

                cout << "Umur (" << b.umur << " tahun): ";
                cin >> b.umur;
                validasiUmur(b.umur);

                cout << "\nData binatang berhasil diubah!" << endl;
                tampilkanBinatang(b, index+1);
            }
        }
        catch (const invalid_argument& e) {
            cout << "\nTERJADI KESALAHAN " << e.what() << endl;
            cout << "Data binatang GAGAL diubah!" << endl;
        }
        catch (const runtime_error& e) {
            cout << "\nTERJADI KESALAHAN " << e.what() << endl;
            cout << "Data binatang GAGAL diubah!" << endl;
        }
        catch (const out_of_range& e) {
            cout << "\nTERJADI KESALAHAN " << e.what() << endl;
        }
        catch (...) {
            cout << "\nTerjadi kesalahan yang tidak diketahui!" << endl;
        }
    } else {
        cout << "\nTidak ada data binatang untuk diubah." << endl;
    }
}

void hapusBinatang(Binatang kebunBinatang[], int *banyak) {
    if(*banyak > 0) {
        int pilihan;
        cout << "\n=== HAPUS BINATANG ===" << endl;
        cout << "1. Hapus berdasarkan nomor urut" << endl;
        cout << "2. Hapus berdasarkan kode hewan" << endl;
        cout << "Pilihan: ";
        cin >> pilihan;
```

```
        int index = -1;
```

4.hasil Output

```
1. Register
2. Login
3. Keluar
Pilihan: 1

=== REGISTRASI ===
Masukkan Nama: erfandi
Masukkan Password: 098

Registrasi Berhasil!
```

```
=== MENU UTAMA ===
1. Register
2. Login
3. Keluar
Pilihan: 2

=== LOGIN ===
Nama: erfandi
Password: 098
```

Gambar output 4.1 register **Gambar output 4.2 login**

```
=== MENU PROGRAM KEBUN BINATANG ===
1. Tampilkan Semua Binatang
2. Tambah Binatang
3. Lihat Detail Binatang
4. Ubah Binatang
5. Hapus Binatang
6. Cari Binatang
7. KEMBALI KE MENU UTAMA
Pilihan: 1

=== DAFTAR SEMUA BINATANG ===
Total: 1 binatang
1. beruang (karnivora)
```

```
=== TAMBAH BINATANG BARU ===
Masukkan nama binatang: beruang
Masukkan jenis (Karnivora/Herbivora/Omnivora): karnivora
Masukkan habitat (Darat/Air/Udara): darat
Masukkan umur (tahun): 2

Binatang berhasil ditambahkan!
```

Gambar output 4.3 tampilkan

```
=== LIHAT DETAIL BINATANG ===
Masukkan nomor binatang (1-1): 1

--- Data Binatang ke-1 ---
Nama     : beruang
Jenis    : karnivora
Habitat  : darat
Umur     : 2 tahun
```

Gambar output 4.4 Tambah

```
Data binatang berhasil diubah!

--- Data Binatang ke-1 ---
Nama     : harimau
Jenis    : karnivora
Habitat  : darat
Umur     : 1 tahun
```

Gambar output 4.5 detail

Gambar output 4.6 ubah

```
Pilihan: 5

=== HAPUS BINATANG ===
Masukkan nomor binatang yang akan dihapus (1-1): 1

Data yang akan dihapus:

--- Data Binatang ke-1 ---
Nama     : harimau
Jenis    : karnivora
Habitat  : darat
Umur     : 1 tahun
```

Gambar output 4.7 hapus

```
=== CARI BINATANG ===
Pilih kriteria pencarian:
1. Cari berdasarkan kode
2. Cari berdasarkan nama
3. Cari berdasarkan jenis
4. Cari berdasarkan habitat
Pilihan:
```

Gambar output 4.8 Cari kriteria

```
=== MENU SORTING BINATANG ===
1.Nama (Z-A)
2.Umur (Termuda ke Tertua)
3.Habitat (A-Z)
4. Kembali
```

```
--- Data Binatang ke-1 ---
Nama     : cheetah
Jenis    : karnivora
Habitat  : darat
Umur     : 2026 tahun

--- Data Binatang ke-2 ---
Nama     : beruang
Jenis    : omnivora
Habitat  : darat
Umur     : 2026 tahun

--- Data Binatang ke-3 ---
Nama     : Angsa
Jenis    : omnivora
```

Gambar output 4.9 sorting

5.1 GIT Add

```
PS C:\Users\Lenovo\OneDrive\praktikum-apl> git add post-test
```

Fungsinya untuk menambah/add file yang sudah kita edit ke staging area

5.2 GIT Commit

```
PS C:\Users\Lenovo\OneDrive\praktikum-apl> git commit -m "up tugas apl"
[main 9ab5336] up tugas apl
 5 files changed, 474 insertions(+)
 create mode 100644 post-test/post-test-apl-3/.vscode/c_cpp_properties.json
 create mode 100644 post-test/post-test-apl-3/.vscode/launch.json
 create mode 100644 post-test/post-test-apl-3/.vscode/settings.json
 create mode 100644 post-test/post-test-apl-3/2509106098-ErfandiMarselianus-PT-3.cpp
 create mode 100644 post-test/post-test-apl-3/2509106098-ErfandiMarselianus-PT-3.exe
```

Fungsi commit adalah untuk menyimpan versi local dari file yang sudah kita add di ikuti dengan ketik pesan agar memudahkan dan memperjelas perubahan apa yang terjadi

5.3 GIT Push

```
PS C:\Users\Lenovo\OneDrive\praktikum-apl> git push -u
Enumerating objects: 12, done.
Counting objects: 100% (12/12), done.
Delta compression using up to 16 threads
Compressing objects: 100% (10/10), done.
Writing objects: 100% (10/10), 457.62 KiB | 7.38 MiB/s, done.
Total 10 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Resolving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/frogyAI/praktikum-apl.git
```

Fungsi nya untuk mengupload file local ke github(global) agar bisa dilihat orang dan memudahkan kita untuk menyimpan file diluar computer